1. מטרת המערכת והיקף המסמך

1.1 הבעיה הפיזית

LayerForge הופך איור בצבעים שטוחים לערימה (stack) של לוחות מודפסים. כל לוח מודפס בחומר אחד בצבע אחד. כשמסתכלים מלפנים, כל נקודה בתמונה מקבלת את הצבע של **הלוח הקדמי ביותר שיש בו חומר בנקודה הזאת**. לכן כדי שצבע מלוח אחורי ייראה, כל הלוחות שלפניו חייבים להיות חלולים (חור) באותה נקודה.

הדרישות שהקוד אוכף בפועל ב־Option B (OBSERVED_IN_FILE בדוח `poc_report.json` ובקוד):

- **היקף משותף:** לכל הלוחות מסגרת חיצונית זהה ברוחב 128 פיקסלים (12.5 מ"מ) סביב שטח יצירה של 200 × 200 מ"מ. גודל הלוח הכולל: 225 × 225 מ"מ.
- **חיבור למסגרת:** כל לוח הוא רכיב 4־קשיר (connected-4) אחד. אין "איים" צפים.
- **רוחב מבני מינימלי:** החלק הנושא של כל לוח מחובר למסגרת דרך מעברים ברוחב של 13 פיקסלים לפחות (1.27 מ"מ, כלומר 1.2 מ"מ אחרי עיגול כלפי מעלה; פרק 6).
- **שחזור מדויק:** כשמרכיבים את הלוחות, התמונה המתקבלת זהה פיקסל־לפיקסל לתמונת היעד השטוחה, ללא "גשרים" נראים.
- **צבעים חוזרים מותרים:** אותו צבע יכול להופיע בכמה לוחות. ב־Option B השחור מופיע פעמיים.
- **מינימום לוחות:** מטרה משנית שמושגת בחיפוש beam. אין לה הוכחת אופטימליות (פרק 8).

1.2 מה המערכת עושה, בשורה אחת לכל שלב

1. מחלצת פלטה אוטומטית מהתמונה המקורית (k-means במרחב CIELAB) וממפה כל פיקסל לצבע הקרוב בפלטה (פרק 4).
2. מקטינה את התמונה לגובה 2048 פיקסלים (200 מ"מ) וממלאת את הצדדים בצבע מסגרת (magenta). AD HOC
3. לפי בקשת המשתמש, מתקנת את האיור באופן ידני וממוקד. ב־Option B: קוצים צהובים דקים בלחי, ופס הכובע שהומר לשחור. AD HOC
4. מחלקת את האזורים ל"תאים" (cells) קטנים ומחליטה לכל תא באיזה עומק (לוח) הוא ייחשף (פרקים 6–8).
5. בונה לכל לוח מסכת חומר פיזית: החלק הגלוי שלו ועוד חומר נסתר שתומך בו (פרקים 7 ו־9).
6. מאמתת כל לוח עם ה־validator הקיים: קשירות ורוחב (פרק 10).
7. הופכת כל מסכה לפוליגונים, מבצעת extrusion ל־1.6 מ"מ ב־CadQuery ומייצאת STEP ו־STL. אחר כך קוראת את הקבצים בחזרה ובודקת אותם (פרק 11).

1.3 מה כלול ומה לא

כלול: הסקריפטים `run_turtle_reveal_planner_poc.py`, `reveal_planner.py`, `fullres_flatten.py` והפונקציות המשותפות שהם קוראים להן: ה־validator מ־`export_reveal_stack_cad.py`, הגדרות הרשת מ־`connectivity_repair.py`, `subtractive_poc.py`, `hidden_width_repair.py` ו־`layer_optimization.py`, ה־extrusion מ־`geometry.py` ו־`exact_index` מ־`experimental_global_planner.py`.

לא כלול: כל אלגוריתם שלא רץ ב־Option B. ביניהם מתכנן ה־CP-SAT (הפונקציה `exact_index` שבקובץ שלו כן שימשה, אבל ה־solver עצמו לא הורץ), מתכנני beam ישנים, ה־flattening הישן, נרמול קווי מתאר ותיקוני רוחב ישנים.

1.4 תשובות קצרות לשאלות המרכזיות

שאלה	תשובה לפי הקוד
מהו המתכנן הפעיל?	`RevealPlanner` במצב `split_regions=True` ("per_cell"). זהו מתכנן beam שעובד על תאים, עם "maximal reveal" כנקודת שבת.
האם נעשה שימוש ב־CP-SAT או ב־AI?	לא. אין solver חיצוני ואין מודל לומד. יש מורפולוגיה של OpenCV, חיפוש beam, ו־k-means (שהוא clustering דטרמיניסטי־סטטיסטי, לא AI).
האם המסלול כולו הוא פקודה אחת?	לא. **GAP** ה־flattening, המסגור, התיקונים הידניים והאימות החוזר של ה־CAD הורצו כקטעי קוד חד־פעמיים. ראה פרק 3.
האם המתכנן משנה את האיור?	לא. הוא רק מחליט עומק לכל תא וכמה חומר נסתר לשמור. השינויים באיור (Option A/B) נעשו ידנית לפני התכנון.